

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

PENGANTAR MIKROKONTROLER

Gloria Gledis Saidui - 5024241096

2026

1 Langkah-Langkah Percobaan

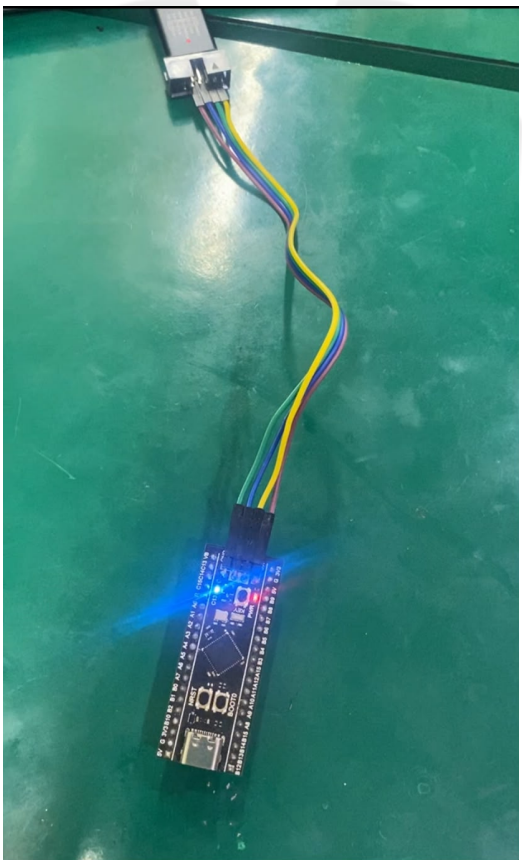
1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Menyiapkan project Modul 1 yang telah dibuat sebelumnya, kemudian membuka file `platformio.ini` dan menyesuaikan konfigurasinya sebagai berikut.

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

2. Menunggu proses refresh konfigurasi PlatformIO hingga selesai sebelum melanjutkan ke tahap berikutnya.

3. Menghubungkan ST-Link ke STM32 Blackpill menggunakan kabel jumper dengan mencocokkan pin SWDIO, SWCLK, GND, dan 3.3V ke pin yang sesuai pada board. Selanjutnya, menghubungkan ST-Link ke komputer melalui USB.



Gambar 1: led pin PC13 nyala

4. Membuka file `src/main.cpp`, kemudian memasukkan kode program Blink berikut.

```
#include <Arduino.h>

#define LED_PIN PC13

```

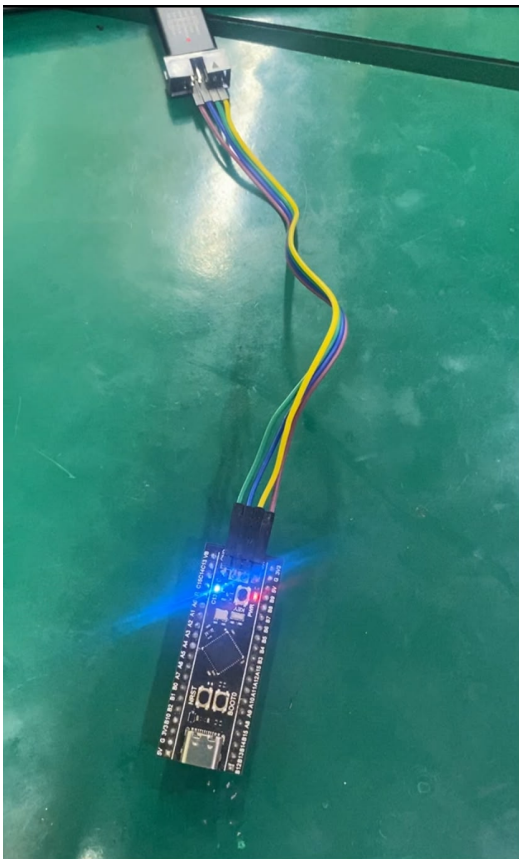
```
void setup() {
    pinMode(LED_PIN, OUTPUT);
}

void loop() {
    digitalWrite(LED_PIN, LOW);
    delay(500);

    digitalWrite(LED_PIN, HIGH);
    delay(500);
}
```

5. Menekan Build untuk mengompilasi program. Jika tidak terdapat kesalahan, melanjutkan dengan menekan Upload untuk mengirimkan program ke board melalui ST-Link.

6. Mengamati LED onboard pada pin PC13. LED akan menyala selama 0,5 detik dan mati selama 0,5 detik secara berulang.



Gambar 2: led pin PC13 nyala

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Menyiapkan project Modul 1 yang telah dibuat sebelumnya, kemudian membuka file `platformio.ini` dan mengganti konfigurasinya menjadi seperti berikut.

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

2. Menunggu proses refresh konfigurasi PlatformIO hingga selesai. Setelah itu, menghubungkan board ESP32 ke komputer menggunakan kabel USB dan memastikan port perangkat telah terdeteksi.

3. Menghapus file `src/main.cpp`, kemudian membuat file baru bernama `src/main.c`. Selanjutnya, memasukkan kode program Blink ESP-IDF berikut.

```c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

void app_main(void) {
    gpio_reset_pin(LED_PIN);
    gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

    while (1) {
        gpio_set_level(LED_PIN, 1);
        vTaskDelay(pdMS_TO_TICKS(500));

        gpio_set_level(LED_PIN, 0);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

4. Menjalankan proses Build untuk mengompilasi program. Jika tidak terdapat kesalahan, melanjutkan dengan menekan Upload untuk mengirimkan program ke ESP32.

5. Mengamati LED onboard pada GPIO2. LED akan menyala selama 0,5 detik dan mati selama 0,5 detik secara berulang.



Gambar 3: led nyala

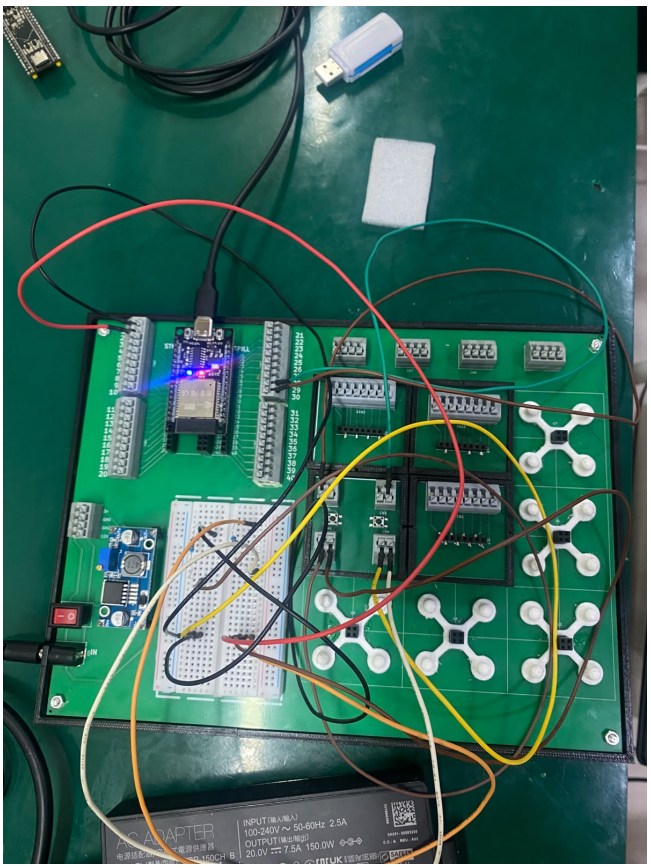
1.3 Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

1. Menyiapkan project Modul 1, kemudian mengubah konfigurasi `platformio.ini` menjadi seperti berikut.

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
```

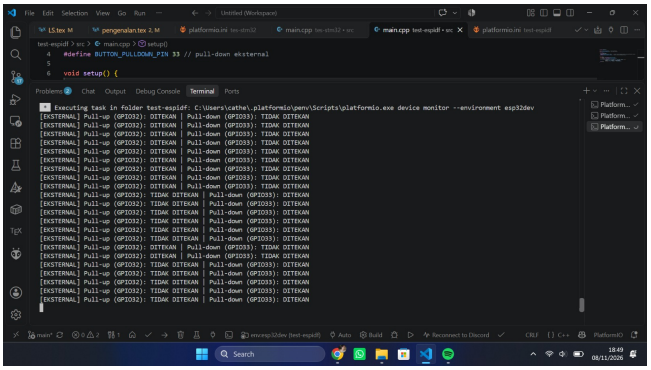
2. Menunggu proses refresh konfigurasi PlatformIO hingga selesai. Selanjutnya, menghapus file `src/main.c` dan membuat kembali file `src/main.cpp`.

3. Memasang dua pushbutton pada GPIO32 dan GPIO33 sesuai skema rangkaian Bagian A. Pushbutton pertama menggunakan resistor pull-up 10kΩ yang dihubungkan ke 3.3V, sedangkan pushbutton kedua menggunakan resistor pull-down 10kΩ yang dihubungkan ke GND.



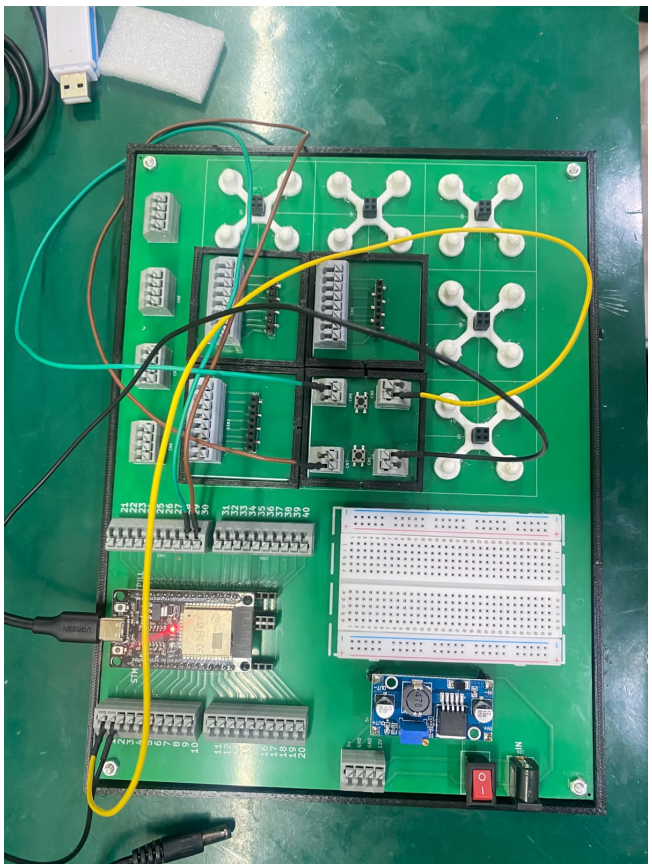
Gambar 4: rangkaian

4. Memasukkan Kode Program A (Eksternal) ke dalam `src/main.cpp`, kemudian menjalankan Build dan Upload. Setelah berhasil, menekan kedua tombol secara bergantian dan mengamati hasil pembacaannya melalui Serial Monitor.



Gambar 5

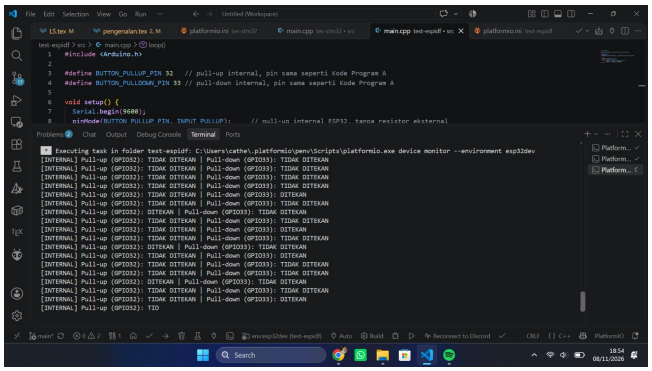
5. Melepas kedua resistor eksternal tanpa mengubah posisi pushbutton. Selanjutnya, mengatur rangkaian sesuai skema Bagian B dengan menghubungkan pushbutton pertama ke GND dan pushbutton kedua ke 3.3V.



Gambar 6: rangkaian

6. Mengganti program dengan Kode Program B (Internal), kemudian melakukan Build dan Upload kembali. Menguji kedua tombol dan mencatat hasil pembacaannya melalui Serial Monitor.

7. Membandingkan hasil pengujian kedua rangkaian. Pada pull-up, kondisi tombol tidak ditekan terbaca HIGH dan saat ditekan terbaca LOW. Sebaliknya, pada pull-down, kondisi tidak ditekan terbaca LOW dan saat ditekan terbaca HIGH.

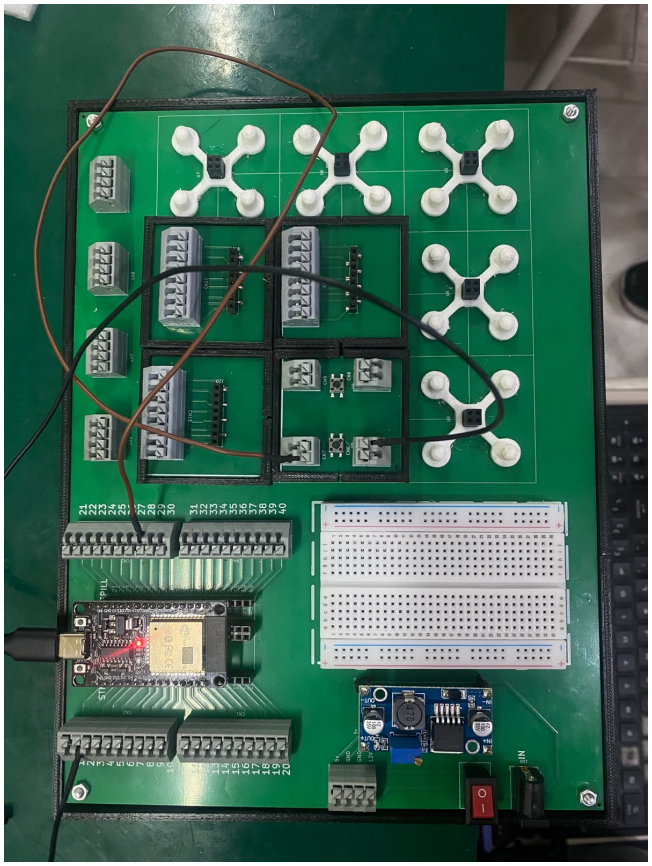


Gambar 7

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

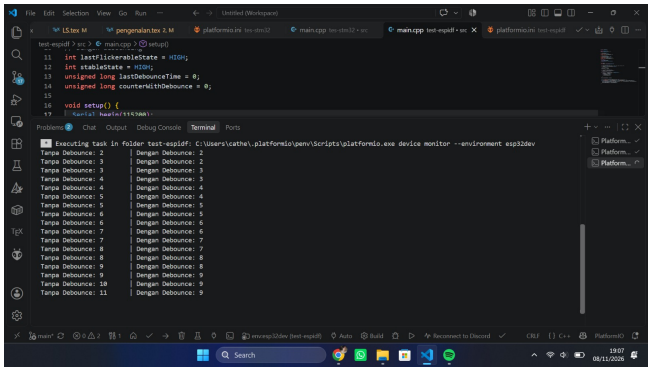
1. Menyiapkan project Modul 1 dengan konfigurasi `platformio.ini` yang sama seperti Percobaan 3.

2. Memasang pushbutton pada GPIO25 dengan menghubungkan satu kaki ke GPIO25 dan kaki lainnya ke GND menggunakan INPUT_PULLUP.



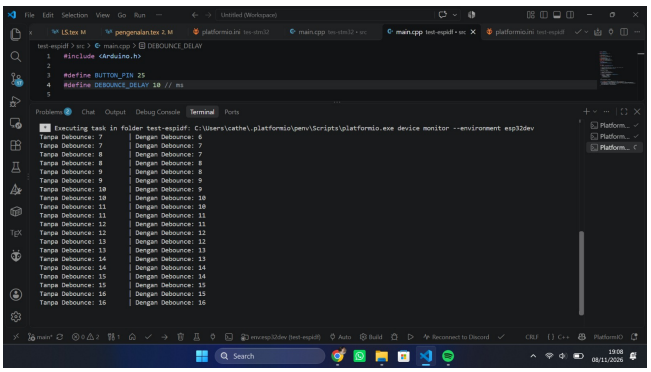
Gambar 8: hasil rangkaian

3. Membuka file `src/main.cpp`, kemudian memasukkan kode program debouncing dengan nilai awal DEBOUNCE_DELAY sebesar 50 ms.

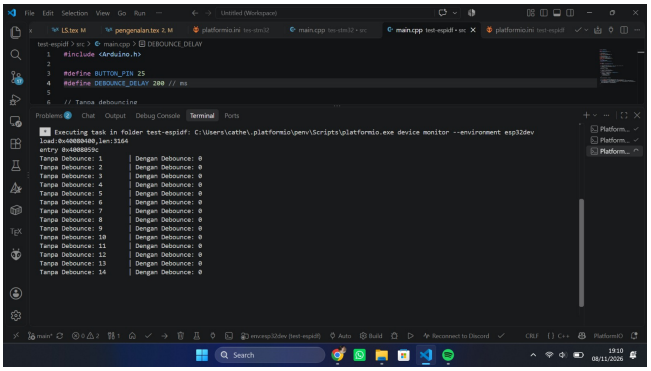


Gambar 9

4. Menjalankan Build dan Upload, kemudian membuka Serial Monitor untuk mengamati perbandingan penghitung tanpa dan dengan debouncing.
5. Menekan tombol secara cepat maupun perlahan, lalu mencatat perubahan nilai kedua penghitung.
6. Mengubah DEBOUNCE_DELAY menjadi 10 ms dan 200 ms secara bergantian. Melakukan Build dan Upload setiap kali nilai diubah, kemudian membandingkan hasilnya.



Gambar 10: 10ms

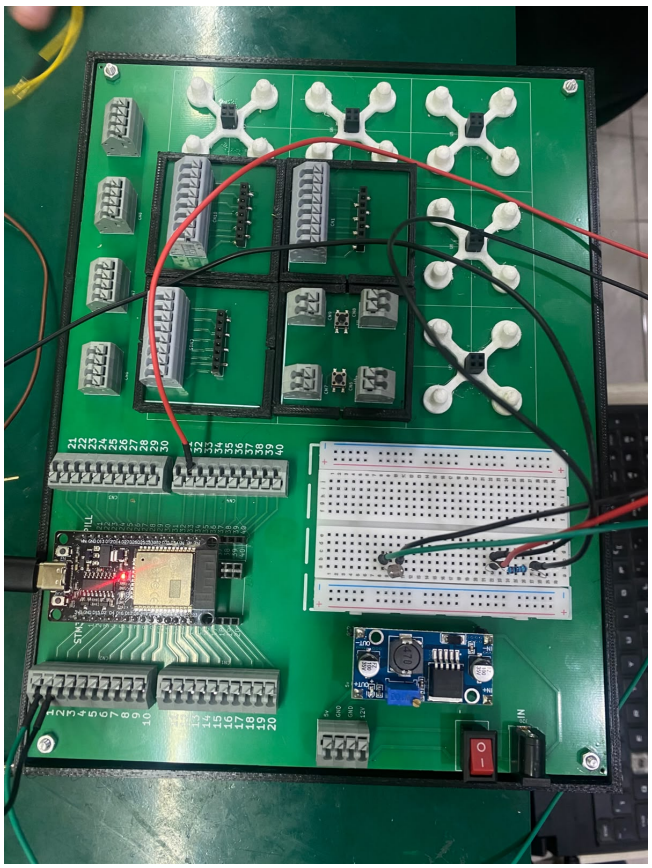


Gambar 11: 200ms

1.5 Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

1. Menyiapkan project Modul 1 dengan konfigurasi `platformio.ini` yang sama seperti Percobaan 3 dan memastikan framework yang digunakan adalah Arduino.

2. Merangkai LDR dan resistor 10kΩ sebagai pembagi tegangan sesuai skema. Menghubungkan titik tengah rangkaian ke GPIO34 pada ESP32.



Gambar 12: rangkaian

3. Membuka file `src/main.cpp`, kemudian memasukkan kode program pembacaan ADC dengan LDR yang telah disediakan.

```cpp
#include <Arduino.h>

#define LDR_PIN 34

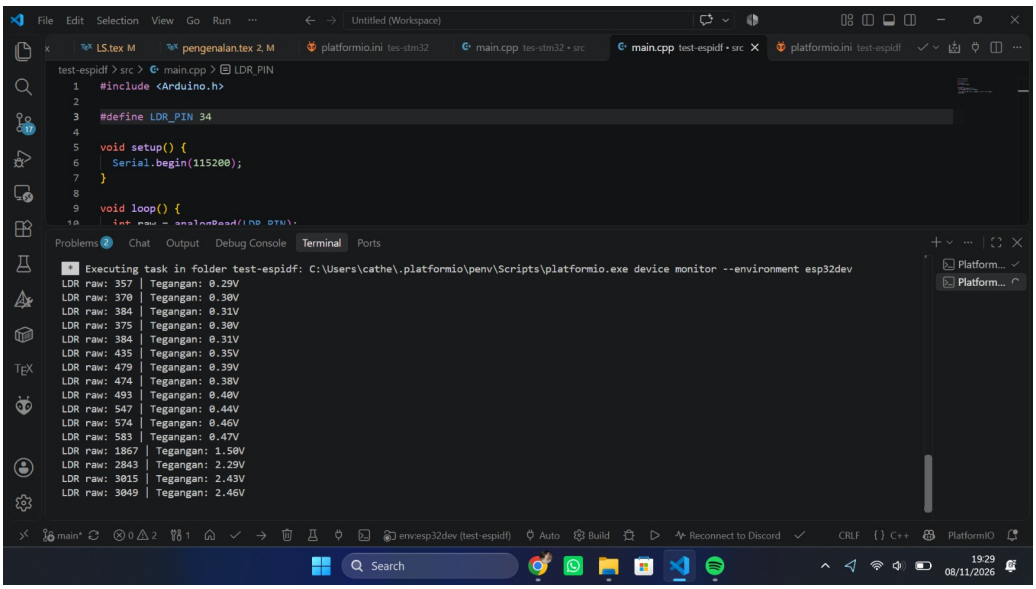
void setup() {
    Serial.begin(115200);
}

void loop() {
    int raw = analogRead(LDR_PIN);
    float voltage = raw * (3.3 / 4095.0);

    Serial.printf("LDR raw: %d | Tegangan: %.2fV\n", raw, voltage);
    delay(300);
}
```

4. Menjalankan Build dan Upload, kemudian membuka Serial Monitor dengan baud ratenya untuk melihat hasil pembacaan ADC.

5. Menutup permukaan LDR menggunakan tangan, kemudian menyinarinya dengan lampu. Mengamati perubahan nilai ADC dan tegangan yang ditampilkan pada Serial Monitor.



Gambar 13: hasil pada serial monitor

2 Analisis Hasil Percobaan

Praktikum Modul 1 membahas penggunaan GPIO pada mikrokontroler STM32 Blackpill dan ESP32 melalui beberapa percobaan, mulai dari pengaturan output digital, pembacaan input digital, hingga pembacaan sinyal analog. Pada Percobaan 1, board STM32 Blackpill berhasil menjalankan program Blink menggunakan framework Arduino dan proses flashing melalui ST-Link. Program menggunakan fungsi `setup()` untuk mengatur pin PC13 sebagai output, sedangkan fungsi `loop()` menjalankan perintah untuk menyalakan dan mematikan LED secara berulang dengan jeda 500 ms. LED onboard berhasil berkedip sesuai interval yang ditentukan. Kondisi ini sesuai dengan teori GPIO digital output. Namun, karena LED PC13 bersifat aktif LOW, pemberian logika LOW menyebabkan LED menyala, sedangkan HIGH membuatnya mati.

Pada Percobaan 2, program Blink diterapkan pada ESP32 menggunakan framework ESP-IDF. Hasil pengujian menunjukkan bahwa LED pada GPIO2 dapat berkedip dengan interval yang sama, yaitu 500 ms. Meskipun memiliki tujuan yang serupa dengan Percobaan 1, kedua framework mempunyai struktur pemrograman yang berbeda. Arduino menggunakan fungsi `setup()` dan `loop()`, sedangkan ESP-IDF menggunakan `app_main()` sebagai titik awal program dengan perulangan `while(1)`. Pengaturan GPIO pada ESP-IDF juga dilakukan secara lebih eksplisit melalui fungsi `gpio_reset_pin()`, `gpio_set_direction()`, dan `gpio_set_level()`. Selain itu, waktu tunda menggunakan `vTaskDelay()` dari FreeRTOS. Proses build pertama membutuhkan waktu lebih lama, sekitar 390 detik, karena komponen framework perlu dikompilasi. Berdasarkan kedua percobaan tersebut, Arduino lebih sederhana dalam penulisan kode, sedangkan ESP-IDF memberikan pengaturan perangkat keras dan pengelolaan task yang lebih terperinci.

Percobaan 3 dilakukan untuk membandingkan penggunaan resistor pull-up dan pull-down eksternal dengan fitur internal ESP32. Hasil pengujian menunjukkan bahwa kedua metode menghasilkan pola pembacaan logika yang sama. Pada GPIO32 dengan konfigurasi pull-up, tombol yang tidak ditekan menghasilkan HIGH, sedangkan saat ditekan menghasilkan LOW. Sebaliknya, GPIO33 dengan konfigurasi pull-down menghasilkan LOW ketika tidak ditekan dan HIGH ketika ditekan. Perbedaan kedua metode terletak pada penggunaan resistor. Konfigurasi eksternal memerlukan resistor

10 kΩ, sedangkan konfigurasi internal memanfaatkan resistor yang tersedia di dalam ESP32 melalui `INPUT_PULLUP` dan `INPUT_PULLDOWN`. Dengan demikian, resistor internal dapat digunakan untuk menjaga kondisi pin agar tidak floating tanpa memerlukan resistor tambahan. Namun, nilai resistansi internal yang lebih besar membuat tarikan logikanya relatif lebih lemah dibandingkan resistor eksternal.

Selama pengujian Percobaan 3, Serial Monitor sempat menampilkan karakter yang tidak terbaca. Permasalahan tersebut disebabkan oleh ketidaksesuaian baud rate antara program dan Serial Monitor. Program menggunakan `Serial.begin(115200)`, sedangkan Serial Monitor masih membaca pada kecepatan 9600 bps. Akibatnya, data yang dikirim melalui komunikasi serial tidak dapat diterjemahkan dengan benar. Setelah baud rate disamakan menjadi 9600, keluaran Serial Monitor dapat terbaca kembali. Permasalahan ini menunjukkan bahwa nilai baud rate pada program harus disesuaikan dengan pengaturan Serial Monitor agar komunikasi berjalan dengan baik.

Pada Percobaan 4, pengujian dilakukan untuk mengetahui pengaruh bouncing pada pushbutton dengan membandingkan penghitung tanpa debouncing dan dengan debouncing. Pada pengaturan 50 ms, penghitung tanpa debouncing mencapai 11, sedangkan penghitung dengan debouncing mencapai 9. Perbedaan dua hitungan tersebut menunjukkan bahwa sebagian perubahan sinyal akibat bouncing berhasil disaring. Ketika waktu debouncing diubah menjadi 10 ms, kedua penghitung mencapai 16 karena waktu penyaringan terlalu singkat untuk menghilangkan seluruh pantulan sinyal. Sementara itu, pada pengaturan 200 ms, penghitung tanpa debouncing mencapai 14, tetapi penghitung dengan debouncing tetap 0. Hal ini terjadi karena tekanan tombol yang dilakukan secara cepat tidak menghasilkan kondisi LOW yang stabil selama 200 ms. Hasil tersebut membuktikan bahwa pemilihan waktu debouncing memengaruhi ketepatan dan kecepatan pembacaan tombol. Nilai 50 ms menjadi pilihan yang cukup seimbang antara kestabilan pembacaan dan respons tombol pada pengujian ini.

Percobaan 5 memperkenalkan penggunaan GPIO sebagai input analog melalui ADC dengan memanfaatkan sensor LDR dan resistor 10 kΩ sebagai pembagi tegangan. Pembacaan dilakukan pada GPIO34 menggunakan fungsi `analogRead()` dengan resolusi 12-bit, sehingga menghasilkan rentang nilai 0–4095. Ketika LDR ditutup, nilai ADC berada pada kisaran 357–583 dengan tegangan sekitar 0,29–0,47 V. Sebaliknya, ketika LDR disinari, nilai ADC meningkat menjadi 1867–3049 dengan tegangan sekitar 1,50–2,46 V. Perubahan tersebut terjadi karena hambatan LDR menurun ketika intensitas cahaya meningkat, sehingga tegangan pada titik tengah pembagi tegangan menjadi lebih besar. Nilai ADC kemudian dikonversi menjadi perkiraan tegangan menggunakan persamaan:

$$V = \frac{\text{raw}}{4095} \times 3,3$$

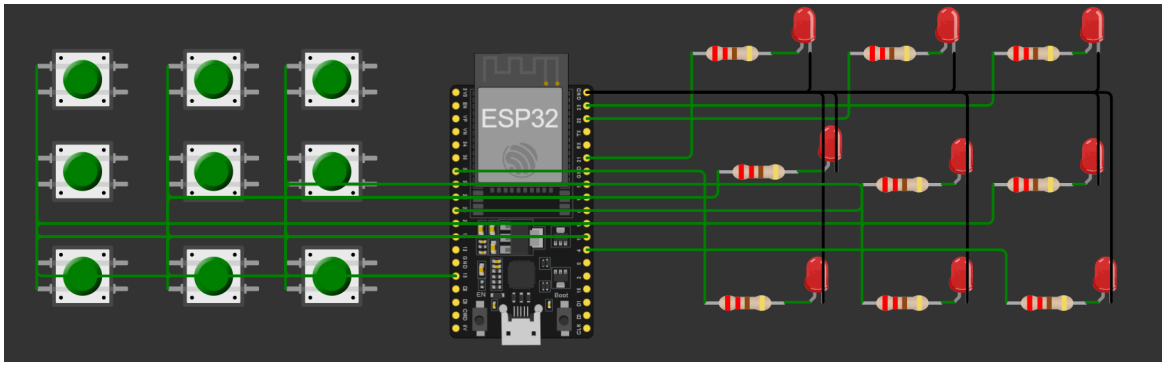
Hasil pengujian sesuai dengan prinsip kerja ADC, yaitu mengubah sinyal analog menjadi nilai digital. Berbeda dengan input digital yang hanya menghasilkan HIGH atau LOW, ADC mampu membaca perubahan tegangan dalam banyak tingkatan. Dengan resolusi 12-bit, ESP32 dapat merepresentasikan hingga 4096 tingkat nilai pembacaan.

Secara keseluruhan, kelima percobaan menunjukkan bahwa GPIO pada mikrokontroler dapat digunakan dalam beberapa mode sesuai kebutuhan sistem. Percobaan 1 dan 2 memperlihatkan penggunaan digital output untuk mengendalikan LED, Percobaan 3 dan 4 membahas digital input untuk membaca tombol beserta pengaruh konfigurasi resistor dan debouncing, sedangkan Percobaan 5 memperkenalkan analog input melalui ADC. Hasil praktikum juga menunjukkan bahwa keberhasilan sistem tidak hanya ditentukan oleh program, tetapi dipengaruhi oleh konfigurasi framework, ketepat-

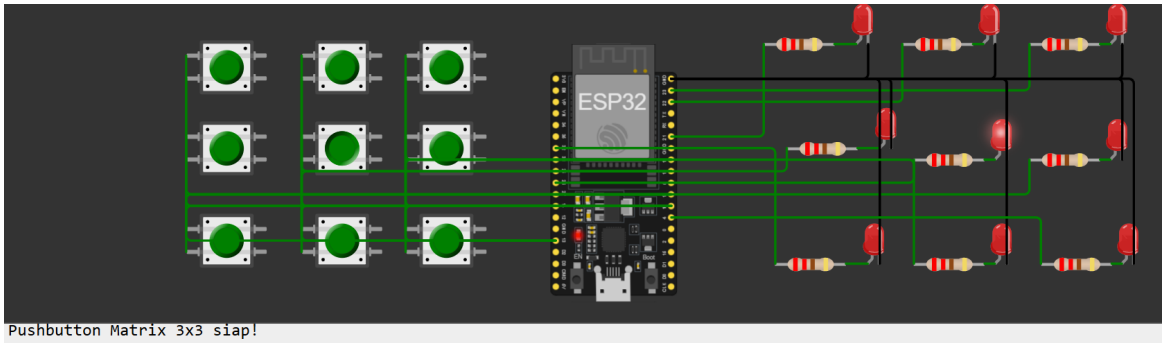
an koneksi rangkaian, pengaturan komunikasi serial, serta pemilihan parameter pembacaan yang sesuai.

3 Hasil Tugas Modul

3.1 Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3



Gambar 14: Rangkaian Simulasi Wokwi



Gambar 15: hasil simulasi

3.2 Source Code

```
#include <Arduino.h>
const uint8_t ROW_PINS[3] = {13, 14, 16};
const uint8_t COL_PINS[3] = {17, 18, 19};

// Pin LED 3x3
const uint8_t LED_PINS[3][3] = {
  {21, 22, 23},
  {25, 26, 27},
  {32, 33, 4}
};

const unsigned long DEBOUNCE_MS = 50;
bool lastReading[3][3];
```

```
bool stableState[3][3];

unsigned long lastDebounceTime[3][3];

void setup() {
  Serial.begin(115200);


  for (int r = 0; r < 3; r++) {
    digitalWrite(ROW_PINS[r], LOW);
    pinMode(ROW_PINS[r], INPUT);
  }


  for (int c = 0; c < 3; c++) {
    pinMode(COL_PINS[c], INPUT_PULLUP);
  }


  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      pinMode(LED_PINS[r][c], OUTPUT);
      digitalWrite(LED_PINS[r][c], LOW);

      lastReading[r][c] = HIGH;
      stableState[r][c] = HIGH;
      lastDebounceTime[r][c] = 0;
    }
  }

  Serial.println("Pushbutton Matrix 3x3 siap!");
  Serial.println("Tekan tombol untuk menyalakan LED.");
}

void loop() {


  for (int r = 0; r < 3; r++) {


    for (int i = 0; i < 3; i++) {
      pinMode(ROW_PINS[i], INPUT);
    }


    digitalWrite(ROW_PINS[r], LOW);
    pinMode(ROW_PINS[r], OUTPUT);

    delayMicroseconds(100);


    for (int c = 0; c < 3; c++) {

      bool reading = digitalRead(COL_PINS[c]);
```

```
68
69
70        if (reading != lastReading[r][c]) {
71          lastReading[r][c] = reading;
72          lastDebounceTime[r][c] = millis();
73        }
74
75
76        if (millis() - lastDebounceTime[r][c] >= DEBOUNCE_MS) {
77
78          if (reading != stableState[r][c]) {
79            stableState[r][c] = reading;
80
81            if (stableState[r][c] == LOW) {
82              Serial.printf(
83                "Tombol (%d,%d) ditekan -> LED (%d,%d) ON\n",
84                r + 1, c + 1, r + 1, c + 1
85              );
86            } else {
87              Serial.printf(
88                "Tombol (%d,%d) dilepas -> LED (%d,%d) OFF\n",
89                r + 1, c + 1, r + 1, c + 1
90              );
91            }
92          }
93        }
94
95
96        digitalWrite(
97          LED_PINS[r][c],
98          stableState[r][c] == LOW ? HIGH : LOW
99        );
100     }
101
102
103     pinMode(ROW_PINS[r], INPUT);
104   }
105 }
```

3.3 Pertanyaan Analisis

1. Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing.

   Jawaban:

   Debouncing tetap diperlukan karena pushbutton merupakan komponen mekanik yang dapat mengalami bouncing, yaitu getaran kontak saat tombol ditekan atau dilepaskan sehingga menghasilkan beberapa perubahan sinyal HIGH dan LOW dalam waktu singkat. Meskipun hanya satu tombol yang ditekan, bouncing dapat menyebabkan ESP32 membaca satu tekanan sebagai beberapa kali penekanan. Oleh karena itu, debouncing digunakan untuk memastikan sinyal

tombol stabil selama 50 ms sebelum dianggap sebagai input yang valid, sehingga LED hanya berubah status sesuai tekanan tombol yang sebenarnya.

2. Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating?

   Jawaban:

   Konfigurasi yang digunakan pada rangkaian adalah pull-up dengan mengaktifkan INPUT_PULLUP pada pin GPIO ESP32. Ketika tombol tidak ditekan, pin GPIO berada pada kondisi HIGH karena ditarik ke tegangan 3,3 V melalui resistor pull-up internal. Sebaliknya, ketika tombol ditekan, pin terhubung ke GND sehingga terbaca LOW. Kondisi pin tidak boleh dibiarkan floating karena tegangan input menjadi tidak menentu akibat gangguan listrik (noise), sehingga ESP32 dapat membaca perubahan HIGH dan LOW secara tidak konsisten. Dengan pull-up, pembacaan tombol menjadi lebih stabil dan dapat diandalkan.

3. Andaikan salah satu Tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?

   Jawaban:

   Digital input hanya membaca dua kondisi logika, yaitu HIGH dan LOW, sehingga hanya dapat menunjukkan apakah tombol ditekan atau tidak. Sementara itu, analog input menggunakan ADC (Analog-to-Digital Converter) untuk mengubah tegangan analog menjadi nilai digital yang dapat diolah oleh ESP32. Pada ESP32, ADC memiliki resolusi 12-bit dengan rentang nilai 0–4095. Jika tombol digantikan dengan LDR, perubahan intensitas cahaya akan memengaruhi resistansi LDR dan tegangan pada rangkaian pembagi tegangan. Perubahan tegangan tersebut kemudian dibaca oleh ADC menjadi nilai yang berbeda-beda. Oleh karena itu, ADC dapat merepresentasikan banyak kondisi intensitas cahaya, bukan hanya HIGH dan LOW.

4 Kesimpulan

Berdasarkan praktikum Modul 1 yang telah dilaksanakan, dapat disimpulkan bahwa GPIO pada mikrokontroler STM32 Blackpill dan ESP32 dapat digunakan sebagai digital output, digital input, maupun analog input sesuai kebutuhan sistem. Melalui program Blink, praktikan memahami cara mengendalikan LED menggunakan framework Arduino dan ESP-IDF, serta mengetahui perbedaan struktur program dan pengaturan GPIO pada kedua framework tersebut.

Pengujian pushbutton menunjukkan bahwa resistor pull-up dan pull-down, baik eksternal maupun internal, dapat menjaga kestabilan kondisi logika input. Selain itu, penerapan debouncing berbasis software mampu mengurangi kesalahan pembacaan akibat bouncing, dengan pemilihan waktu tunda yang perlu disesuaikan agar tombol tetap responsif.

Pada pembacaan LDR, ADC ESP32 berhasil mengubah perubahan tegangan analog menjadi nilai digital yang dapat diamati melalui Serial Monitor. Hasil pengukuran menunjukkan bahwa nilai ADC meningkat ketika intensitas cahaya bertambah. Secara umum, hasil praktikum sesuai dengan teori

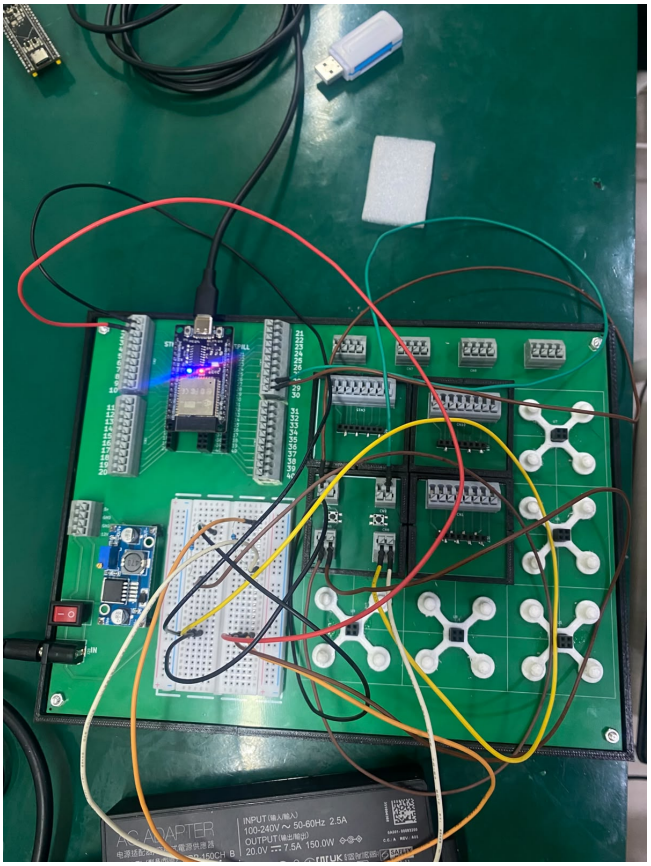
yang dipelajari dan memberikan pemahaman mengenai konfigurasi GPIO, penggunaan framework, pembacaan sensor, serta faktor-faktor yang memengaruhi kerja sistem tertanam.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 16



Gambar 17: rangkaian percobaan 3